Contents

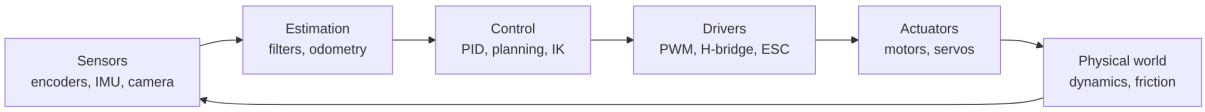
Robotics Core Guide **1**
How to use this guide (and escape tutorial hell) . . . 1
Math I: Vectors, trig, matrices and transforms . . . 2
Math II: Essential calculus and differential equations . . . 4
Physics: Kinematics, dynamics, torque, energy, friction . . . 6
Electronics I: Ohm's law, power, circuits, batteries . . . 8
Electronics II: Signals, sensors, ADC, PWM, filtering . . . 10
Actuators: motors, servos and drivers . . . 13
Mechanical engineering: gears, stress, structures, materials . . . 16
Control: feedback, PID, tuning, state estimation . . . 18
Kinematics: forward and inverse kinematics for arms and legs . . . 21
Mobile robots, legged locomotion and drones . . . 24
Embedded software and system integration . . . 26
Formulas to memorize, project ladder, further reading . . . 28

Robotics Core Guide

Sep 22, 2026 · @j-bsky

How to use this guide (and escape tutorial hell)

Robotics is one loop repeated at every scale: **sense → decide → act**, with physics in between. Every topic below is a tool for making one part of that loop work; learn each one as "what question does this answer" rather than "what formula do I plug in".



The loop closes through the physical world, which is why every section here ends up connecting to control.

Tutorial hell is following steps without a model of *why* they work, so nothing transfers to your own project. The cure is not more tutorials; it is three habits:

1. **Estimate before you build.** Before ordering a motor, compute the torque you need (Section 4 and 8). If the number surprises you, you have just learned something a tutorial never would.
2. **Change one thing and predict the result.** Double the gear ratio; predict speed halves and torque doubles; test it. Prediction-then-measurement is the fastest way to build intuition.
3. **Own a scrappy project end to end.** One wheeled robot that you designed, wired, programmed and tuned yourself teaches more than ten kits. Section 13 gives a ladder of such projects.

How the three documents fit together:

Document	Use it when
This guide	You need the concept, the intuition and the formula
Exercise book	You need the formula to be automatic (drill until you stop looking it up)

Document	Use it when
Parts list	You are about to buy or design something and need real names, materials and prices

Formulas marked **memorize** in this guide are the ones drilled in the exercise book. Everything else you should be able to derive or look up.

Math I: Vectors, trig, matrices and transforms

Almost every robotics calculation is "where is this thing, in which frame, and how do I convert". Vectors carry position and force; matrices convert between frames. Get fluent here and kinematics (Section 10) becomes bookkeeping.

Vectors

A vector is a magnitude plus a direction, written as components in some frame: **v** = (vx, vy, vz). Two operations do 90% of the work.

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}||\mathbf{b}| \cos\theta$$

The dot product (**memorize**) answers "how much of **a** points along **b**". Zero means perpendicular. Use it for projecting a force onto a direction, checking alignment, and computing work (F·d).

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y,\ a_z b_x - a_x b_z,\ a_x b_y - a_y b_x), \quad |\mathbf{a} \times \mathbf{b}| = |\mathbf{a}||\mathbf{b}| \sin\theta$$

The cross product (**memorize**) gives a vector perpendicular to both, with the right-hand rule. Torque is **r × F**; angular velocity relates to linear velocity by **v = ω × r**. Metaphor: the dot product is a shadow, the cross product is a doorknob (turning about an axis perpendicular to both your push and the lever).

Magnitude and unit vector:

$$|\mathbf{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}, \qquad \hat{\mathbf{v}} = \mathbf{v}/|\mathbf{v}|$$

Trigonometry you will use daily

Tool	Formula	Robotics use
SOH-CAH-TOA	sin = opp/hyp, cos = adj/hyp, tan = opp/adj	Resolving a link length into x and y
Pythagorean identity	$\sin^2\theta + \cos^2\theta = 1$	Sanity checks, normalizing rotations
Law of cosines (**memorize**)	$c^2 = a^2 + b^2 - 2ab\cdot\cos C$	Two-link inverse kinematics
atan2(y, x) (**memorize**)	Angle of the vector (x, y), correct in all four quadrants	Heading from odometry, IK base angle
Small-angle	$\sin\theta \approx \theta$, $\cos\theta \approx 1 - \theta^2/2$ (θ in radians)	Linearizing pendulums and balancing robots

Always work in **radians** in code and physics (ω in rad/s, arc length s = rθ). Convert only for display. Never use plain atan for headings: atan2 is the one that knows which quadrant you are in.

Matrices

A matrix is a machine that turns one vector into another. Multiplication is row-times-column; the product **AB** means "apply **B** first, then **A**". Order matters: **AB ≠ BA** in general, which is exactly why rotating then translating differs from translating then rotating.

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc, \qquad \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

A determinant of zero means the matrix squashes space flat and cannot be inverted. In robotics this is a **singularity**: a Jacobian with zero determinant means the arm has lost a direction of motion (fully stretched out, for example).

Rotation matrices

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

The 2D rotation (**memorize**) turns a vector counter-clockwise by θ. Its inverse is its transpose ($R^{-1} = R^{T}$) because rotations preserve length; this is true in 3D too and saves you an inversion. The 3D versions about each axis:

$$R_z(\theta) = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}, \; R_y(\theta) = \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix}, \; R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{bmatrix}$$

with c = cos θ, s = sin θ. Composite orientations (roll, pitch, yaw) are products of these; the order you multiply them in is a convention you must state. Quaternions (a 4-number alternative) avoid gimbal lock and are what IMUs and flight controllers output; you rarely need to hand-compute them, but know they exist and that unit length must be maintained.

Homogeneous transforms

A 4×4 matrix combines rotation and translation so one multiplication moves a point from frame B to frame A:

$${}^{A}T_{B} = \begin{bmatrix} R_{3\times3} & \mathbf{p}_{3\times1} \\ 0\;0\;0 & 1 \end{bmatrix}, \qquad {}^{A}\mathbf{x} = {}^{A}T_{B}\,{}^{B}\mathbf{x}$$

Chain them along a robot: T_base→tool = $T_{01}\,T_{12}\,T_{23}$ … (**memorize** the structure). Reading the superscripts as "from B to A" and checking that inner indices cancel (^{A}T_B ^{B}T_C = ^{A}T_C) catches most bugs. The inverse is cheap:

$$T^{-1} = \begin{bmatrix} R^{T} & -R^{T}\mathbf{p} \\ 0 & 1 \end{bmatrix}$$



Each arrow is one 4×4 matrix; the tool's world position is the product of all of them.

Frames of reference: the mental model

Every number has a frame. A camera says "the cup is 0.3 m ahead" in the camera frame; the arm needs it in the base frame; the wheel encoder reports in the robot frame while the map is in the world frame. When a robot behaves strangely, the first question is always: **which frame is this quantity in?**

Math II: Essential calculus and differential equations

Calculus in robotics is mostly two ideas: a derivative is a **rate** (velocity from position, current from charge) and an integral is an **accumulation** (position from velocity, energy from power). Your microcontroller does both numerically every loop; understanding the continuous version tells you when the numeric version will lie.

Derivatives

$$\frac{d}{dt}x(t) = \lim_{\Delta t \to 0} \frac{x(t+\Delta t) - x(t)}{\Delta t}$$

Rules you need cold (**memorize**):

Rule	Formula
Power	d/dt (t^n) = n·t^{n-1}
Exponential	d/dt (e^{at}) = a·e^{at}
Trig	d/dt sin(ωt) = ω cos(ωt); d/dt cos(ωt) = −ω sin(ωt)
Product	(uv)' = u'v + uv'
Chain	d/dt f(g(t)) = f'(g(t))·g'(t)

The chain rule is the Jacobian in disguise: if the end-effector position depends on joint angles and joint angles depend on time, end-effector velocity is (∂x/∂θ)·(dθ/dt). Section 10 builds on exactly this.

Partial derivatives (∂) hold every variable but one fixed. The **gradient** ∇f points uphill; gradient descent (step against it) is how numerical IK and most learning-based control works.

Integrals

$$\int_{t_0}^{t_1} v(t)\,dt = x(t_1) - x(t_0), \qquad \int t^n dt = \frac{t^{n+1}}{n+1} + C$$

The fundamental theorem says integration undoes differentiation. In code the integral of a signal is a running sum: `x += v * dt`. This is also why an IMU cannot give position for long: integrating a slightly biased acceleration twice makes the error grow with t^2, which is the physics behind "drift".

Numerical versions (what your loop actually runs)

$$\dot{x}_k \approx \frac{x_k - x_{k-1}}{\Delta t}, \qquad X_k = X_{k-1} + x_k\,\Delta t$$

(**memorize** both.) The derivative amplifies noise: a 1-count jitter on an encoder read at 1 kHz becomes a 1000 count/s velocity spike. Filter before differentiating (Section 6), or measure velocity by timing between encoder edges at low speed.

Taylor series and linearization

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

Every nonlinear system looks linear if you zoom in. A balancing robot is a nonlinear pendulum, but near upright sin θ ≈ θ, and suddenly a PID controller designed for a linear system works. Knowing where the approximation breaks (about ±15° for sin θ ≈ θ within 1%) tells you where your controller will start to misbehave.

Differential equations that describe robots

A first-order ODE with an exponential solution shows up everywhere: motor current rising, a capacitor charging, a low-pass filter settling.

$$\tau\frac{dy}{dt} + y = u \quad \Rightarrow \quad y(t) = u\,(1 - e^{-t/\tau})$$

(**memorize**.) τ is the **time constant**: 63% of the way at t = τ, 95% at 3τ, 98% at 4τ. If your motor has an electrical time constant of 2 ms and a mechanical one of 50 ms, you know the current settles long before the speed does, which is why current loops run faster than speed loops.

The second-order ODE describes anything with mass and a spring or a P-controller:

$$m\ddot{x} + c\dot{x} + kx = F, \qquad \omega_n = \sqrt{k/m}, \quad \zeta = \frac{c}{2\sqrt{km}}$$

ω_n is the natural frequency, ζ the damping ratio (**memorize** both). ζ < 1 oscillates, ζ = 1 is the fastest response without overshoot, ζ > 1 is sluggish. When you tune a PID gain and the arm starts bouncing, you have lowered ζ; adding derivative gain adds c and raises it.

ζ	Behaviour	Overshoot
0.2	Rings many times	~53%
0.5	A few oscillations	~16%
0.7	Fast, tiny overshoot (usual target)	~5%
1.0	Critically damped, no overshoot	0%
2.0	Slow, over-damped	0%

What to skip for now

Laplace transforms and transfer functions are the formal language of control theory; useful later but not needed to build working robots. Statistics beyond mean, variance and the Gaussian (needed for Kalman filters) can wait too.

Physics: Kinematics, dynamics, torque, energy, friction

Physics is what decides whether the motor you picked will actually move the robot. Three questions cover most designs: how much force or torque, how much power, and how much energy (battery).

Kinematics: describing motion

$$v = v_0 + at, \qquad x = x_0 + v_0 t + \tfrac{1}{2}at^2, \qquad v^2 = v_0^2 + 2a(x - x_0)$$

(**memorize**.) Rotational motion uses the same equations with θ, ω, α replacing x, v, a. The bridge between the two (**memorize**):

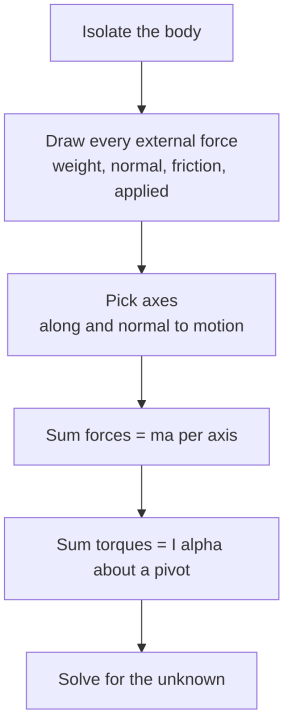
$$v = \omega r, \qquad a = \alpha r, \qquad \omega\,[\text{rad/s}] = \frac{2\pi \cdot \text{RPM}}{60}$$

A 65 mm wheel at 200 RPM: ω = 20.9 rad/s, v = 20.9 × 0.0325 = 0.68 m/s. Do this calculation for every drivetrain before buying motors.

Newton's laws and free-body diagrams

$$\sum \mathbf{F} = m\mathbf{a}, \qquad \sum \tau = I\alpha$$

(**memorize**.) The habit that separates engineers from tinkerers is the **free-body diagram**: draw the object alone, draw every force on it (weight mg down, normal force, friction, motor thrust), then sum. Weight is W = mg with g = 9.81 m/s².



The same six steps solve a climbing robot, a lifting arm and a balancing bot.

Torque

$$\tau = rF\sin\theta = \mathbf{r}\times\mathbf{F}$$

(**memorize**.) Torque is force times lever arm. Only the component of force perpendicular to the lever counts. A 1 kg mass held at the end of a 0.3 m horizontal arm demands 1 × 9.81 × 0.3 = 2.94 N·m at the shoulder, and that is before the arm's own weight and any acceleration. Manufacturers quote servo torque in kg·cm: 1 kg·cm = 0.0981 N·m, so a "20 kg·cm" servo delivers about 1.96 N·m at stall.

Moment of inertia

I is the rotational equivalent of mass: how hard it is to change spin. It scales with mass × radius², so mass far from the axis costs far more than mass near it.

Shape (axis through center)	I
Point mass at radius r	m r²
Solid disc or cylinder	½ m r²
Thin rod, axis at end	⅓ m L²
Thin rod, axis at middle	1/12 m L²
Solid sphere	⅖ m r²

Parallel-axis theorem (**memorize**): I about any axis = I_center + m d², with d the distance between the axes. Rule of thumb for arms: keep motors close to the base and links light; a gram at the wrist costs more than a gram at the shoulder.

Work, energy, power

$$W = Fd\cos\theta, \qquad KE = \tfrac{1}{2}mv^2 = \tfrac{1}{2}I\omega^2, \qquad PE = mgh$$

$$P = Fv = \tau\omega \quad [\mathrm{W}], \qquad E = Pt$$

(**memorize** all.) Power is what sizes the motor; energy is what sizes the battery. A robot that must lift 2 kg by 0.5 m in 1 s needs P = mgh/t = 9.81 W of mechanical output. Motors are perhaps 60–80% efficient and drivers 90–95%, so budget roughly 15 W electrical. Efficiency (**memorize**):

$$\eta = \frac{P_{out}}{P_{in}}, \qquad \eta_{total} = \eta_1\,\eta_2\,\eta_3\cdots$$

Efficiencies multiply along a chain, so a 90% driver, 75% motor and 80% gearbox deliver only 54% of the battery power to the wheel.

Friction

$$F_f \le \mu_s N \ (\text{static}), \qquad F_f = \mu_k N \ (\text{kinetic})$$

(**memorize**.) Friction is proportional to the normal force, not contact area. Static friction is larger than kinetic, which is why a wheel that starts to slip suddenly loses grip. For a wheeled robot the maximum tractive force is μ·(weight on driven wheels); torque beyond that just spins

the tires. Typical μ: rubber on concrete 0.6–0.9, rubber on smooth tile 0.4–0.6, plastic on plastic 0.2–0.4, steel on steel lubricated 0.1.

Rolling resistance is far smaller (coefficient 0.01–0.03 for hard wheels), but it is always there: a 5 kg robot on 0.02 needs about 1 N just to keep rolling on flat ground.

Worked example: sizing a drive motor

A 3 kg robot must climb a 15° ramp at 0.5 m/s on 65 mm wheels, two driven wheels.

1. Force along the ramp: mg sin 15° = 3 × 9.81 × 0.259 = 7.6 N, plus rolling resistance ≈ 0.6 N, total 8.2 N.
2. Torque at the wheels: F × r = 8.2 × 0.0325 = 0.27 N·m, so 0.13 N·m per motor.
3. Speed: ω = v/r = 15.4 rad/s ≈ 147 RPM.
4. Power: P = τω = 0.27 × 15.4 = 4.1 W mechanical, so about 7 W electrical for both motors.
5. Margin: pick a motor that gives 0.13 N·m at roughly half its stall torque (Section 7 explains why), i.e. stall ≥ 0.26 N·m at ≥ 150 RPM loaded.

Doing this by hand once makes every motor datasheet readable.

Electronics I: Ohm's law, power, circuits, batteries

A robot's electrical system is a plumbing problem: voltage is pressure, current is flow, resistance is pipe narrowness. Most fried boards come from ignoring one of three numbers: how many amps, at what voltage, for how long.

Ohm's law and power

$$V = IR, \qquad P = VI = I^2R = \frac{V^2}{R}$$

(**memorize**.) I²R is the one that burns things: doubling current quadruples heat in a wire, a MOSFET or a motor winding. A 5 A motor through a 0.1 Ω trace dissipates 2.5 W in the trace alone.

Series and parallel

$$R_{series} = R_1 + R_2 + \cdots, \qquad \frac{1}{R_{parallel}} = \frac{1}{R_1} + \frac{1}{R_2} + \cdots$$

$$V_{out} = V_{in}\frac{R_2}{R_1 + R_2} \quad \text{(voltage divider)}$$

(**memorize** all three.) The voltage divider is how you read a 12 V battery with a 3.3 V ADC (R_1 = 30 kΩ, R_2 = 10 kΩ gives 3 V at 12 V) and how a potentiometer reports a joint angle. Kirchhoff's laws are the bookkeeping behind these: current into a node equals current out; voltages around any loop sum to zero.

Capacitors and inductors

$$I = C\frac{dV}{dt}, \qquad V = L\frac{dI}{dt}, \qquad \tau_{RC} = RC, \qquad \tau_{RL} = L/R$$

A capacitor resists sudden voltage change (it is a small local reservoir); an inductor resists sudden current change. Every motor is an inductor, so switching it off abruptly makes V = L·dI/dt spike to hundreds of volts. That is why a **flyback diode** across any motor or relay coil is mandatory, and why decoupling capacitors (100 nF close to every chip, 10–100 µF near the supply input) belong on every board: they supply the brief current spikes the battery wires are too slow to deliver.

Semiconductors

Part	What it does	Robotics use
Diode	Lets current flow one way; drops ~0.7 V (Si) or ~0.3 V (Schottky)	Flyback protection, reverse-polarity protection
BJT transistor	Small base current controls a larger collector current	Low-side switching of small loads
MOSFET	Gate voltage controls drain current, near-zero gate current	Motor switching, H-bridges, high-current loads
Voltage regulator	Holds an output voltage steady	Powering logic from the battery

For a MOSFET, check three datasheet numbers: V_GS(th) and the R_DS(on) at your gate voltage (a "logic-level" part is fully on at 3.3–5 V; a standard part may need 10 V), I_D continuous, and V_DS above your battery voltage with margin. Power in the switch is $I^2 \cdot R_{DS(on)}$: a 20 mΩ part at 10 A dissipates 2 W and needs a heatsink or copper pour.

Regulators: linear vs switching

A linear regulator (7805, AMS1117) burns the difference as heat: P = (V_in − V_out) × I. From 12 V to 5 V at 1 A that is 7 W wasted and a very hot part. A buck converter (switching) is 85–95% efficient at the cost of some noise; use buck for anything over about 100 mA. Boost converters step up (3.7 V cell to 5 V); a UBEC is a buck built for hobby-robot voltages.

Batteries

$$t\,[\mathrm{h}] \approx \frac{\text{capacity [Ah]}}{\text{average current [A]}}, \qquad E\,[\mathrm{Wh}] = V \times \mathrm{Ah}$$

(**memorize**.) A 3S LiPo (3 cells in series) is nominally 11.1 V, 12.6 V full, and should never be discharged below ~3.0–3.2 V per cell. A 2200 mAh pack rated "30C" can deliver 30 × 2.2 = 66 A briefly. Chemistry comparison:

Chemistry	Nominal V/cell	Energy density	Notes
LiPo	3.7	High (150–250 Wh/kg)	Highest current, needs balance charging and care
Li-ion 18650	3.6–3.7	High (200–260 Wh/kg)	Safer packaging, lower peak current per cell
LiFePO4	3.2	Medium (90–160 Wh/kg)	Very safe, long life, flat discharge curve
NiMH	1.2	Low (60–120 Wh/kg)	Tolerant, cheap, sags under load
Alkaline	1.5	Low, non-rechargeable	Prototyping only

Always fuse the main battery lead and put motors on their own supply rail (or at least their own regulator) from the microcontroller; brown-outs during motor stall are the most common "random reset" bug.

Wire gauge and current

AWG	Continuous current (chassis wiring, short runs)	Typical use
22	~3 A	Signals, small servos
18	~10 A	Small drive motors
14	~20 A	Battery to power distribution
12	~30 A	Large brushless drives

Voltage drop along a wire (V = IR) matters at high current: 1 m of 22 AWG (≈0.05 Ω) at 5 A loses 0.25 V.

Grounding and noise

All grounds must connect, but motor return current should not flow through the same trace as sensor signals. Use a star ground or separate power and logic ground planes joined at one point. Twist motor leads, keep encoder wires away from them, and add a 100 nF capacitor across brushed-motor terminals to suppress brush noise.

Electronics II: Signals, sensors, ADC, PWM, filtering

Sensors turn physics into voltages; the microcontroller turns voltages into numbers; you turn numbers back into meaning. Each conversion loses something, and knowing what is lost is the whole skill.

Analog to digital

$$\text{code} = \left\lfloor \frac{V_{in}}{V_{ref}} \left(2^N - 1\right) \right\rfloor, \qquad \text{resolution} = \frac{V_{ref}}{2^N}$$

(**memorize**.) A 12-bit ADC with a 3.3 V reference resolves 0.8 mV per count; a 10-bit one, 3.2 mV. Noise on the reference or the input directly becomes noise in the reading, so filter the reference and keep analog wires short.

Sampling: to see a signal of frequency f you must sample at more than 2f (Nyquist). Sample slower and the signal *aliases* into a false low frequency; a 1 kHz motor whine sampled at 900 Hz looks like a 100 Hz wobble. Sample fast enough, or low-pass filter before the ADC.

PWM

$$\text{duty} = \frac{t_{on}}{T}, \qquad V_{avg} = \text{duty} \times V_{supply}$$

(**memorize**.) Pulse-width modulation switches a pin fully on and off quickly; the motor's inductance and inertia average it. Frequency matters:

PWM frequency	Effect
50 Hz, 1–2 ms pulse	Hobby servo command (position ∝ pulse width, not duty)
490 Hz – 1 kHz	Audible whine on brushed motors, fine for LEDs
16–25 kHz	Above hearing; standard for motor drivers
400 Hz – 8 kHz (OneShot, DShot digital)	ESC protocols for brushless drones

Note the servo case: a hobby servo reads pulse *width* (1000–2000 µs, 1500 µs center), refreshed every 20 ms. The duty cycle is incidental.

Digital communication buses

Bus	Wires	Speed	Distance	Typical devices
UART	2 (TX, RX)	9.6 k – 3 Mbaud	Meters	GPS, lidar, serial servos, debug
I²C	2 shared (SDA, SCL)	100–400 kHz (1 MHz fast+)	< 1 m	IMU, ToF distance, OLED, many sensors
SPI	4 (MOSI, MISO, SCK, CS)	1–50 MHz	< 0.5 m	Encoders, ADCs, displays, radio modules
CAN	2 differential	up to 1 Mbit/s	40+ m	Motor controllers, multi-board robots

I²C needs pull-up resistors (2.2–4.7 kΩ) and every device a unique address; SPI needs one chip-select line per device. CAN is the choice when many boards or long wires are involved because it is differential and noise-immune.

Sensors you will actually use

Sensor	Measures	Output	Gotcha
Quadrature encoder	Shaft angle and direction	Two square waves A/B, 90° apart	Count edges in an interrupt; noise causes miscounts
Potentiometer	Absolute angle	Analog voltage	Wears out; limited travel
Magnetic encoder (AS5600, AS5048)	Absolute angle, 12–14 bit	I²C/SPI/PWM	Needs a diametric magnet centered on the shaft
IMU (MPU-6050, BNO055, ICM-42688)	Angular rate + acceleration (+ magnetometer)	I²C/SPI	Gyro drifts, accelerometer is noisy; fuse them
Ultrasonic (HC-SR04)	Distance 2–400 cm	Echo pulse width	Soft or angled surfaces vanish; 40 ms per ping
ToF laser (VL53L0X/L1X)	Distance up to ~4 m	I²C	Sunlight and glass confuse it
2D lidar (RPLidar, LD19)	360° range scan	UART	Cost, black surfaces absorb IR
Current sensor (INA219, ACS712)	Motor current	I²C / analog	Use for stall detection and torque estimation

Sensor	Measures	Output	Gotcha
Limit switch / hall switch	End-of-travel	Digital	Debounce it (5–20 ms)
Camera	Everything, slowly	USB/CSI	Needs a real computer, not a microcontroller

Quadrature decoding (**memorize** the idea): with N pulses per revolution on each channel, counting every edge on both channels gives 4N counts per revolution. Angle = counts × 2π / (4N × gear ratio).

Filtering

The exponential moving average is the workhorse low-pass filter (**memorize**):

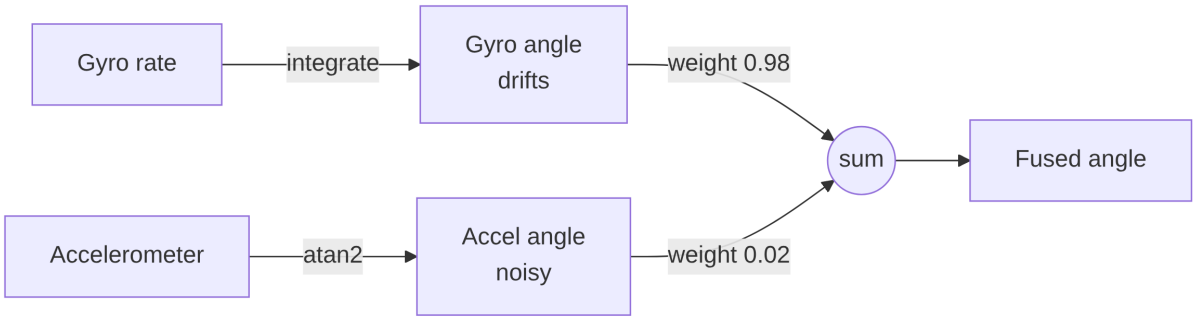
$$y_k = \alpha x_k + (1-\alpha)\, y_{k-1}, \qquad \alpha = \frac{\Delta t}{\tau + \Delta t}$$

α near 1 trusts new samples (fast, noisy); α near 0 trusts history (smooth, laggy). It is the discrete form of the first-order ODE from Section 3, with cutoff f_c = 1/(2πτ). Every filter adds delay, and delay in a feedback loop destabilizes it, so filter no more than you must.

The **complementary filter** fuses a gyro (good short term, drifts long term) with an accelerometer (noisy short term, honest long term) into one angle (**memorize**):

$$\theta_k = \alpha\,(\theta_{k-1} + \omega_{gyro}\Delta t) + (1-\alpha)\,\theta_{accel}, \quad \alpha \approx 0.98$$

θ_accel = atan2(a_y, a_z) for a single tilt axis. This one line is enough for a balancing robot or a drone's attitude estimate; a Kalman filter is its statistically optimal cousin (Section 9).



Read the diagram as: the gyro sets the shape of the signal, the accelerometer slowly corrects where it sits.

Signal integrity checklist

- Debounce mechanical switches in software or with an RC filter.
- Use interrupts, not polling, for encoders above a few hundred counts per second.
- Level-shift between 5 V sensors and 3.3 V microcontrollers; many 3.3 V pins are not 5 V tolerant.
- Put a 100 nF capacitor across analog sensor outputs to kill PWM pickup.

Actuators: motors, servos and drivers

Pick the motor type by what you need to control (speed, position, or both) and how much torque, then pick the driver to match the current. Most beginner failures are undersized motors or undersized drivers, both fixable with the arithmetic below.

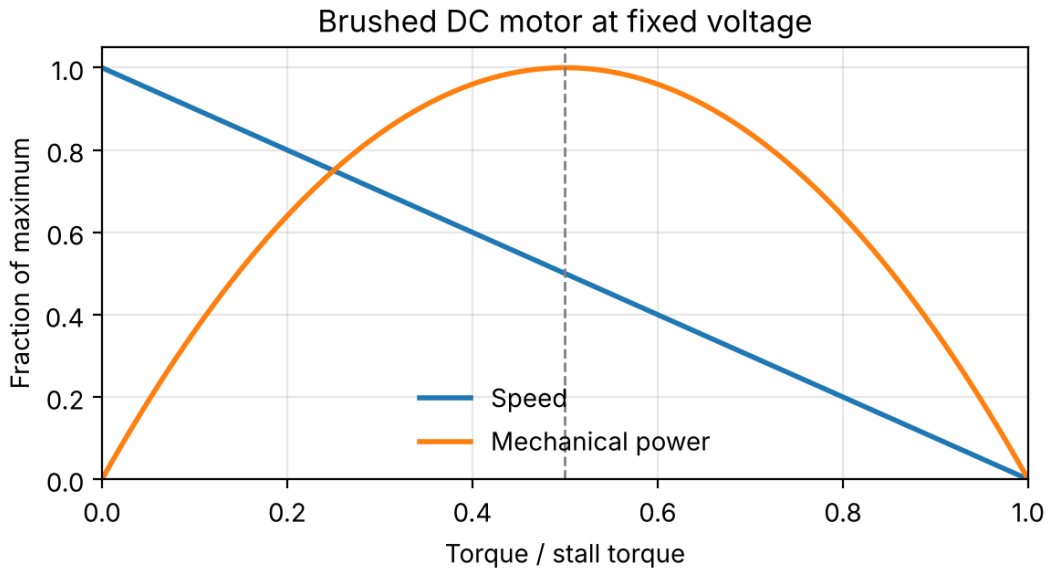
Brushed DC motor model

$$V = IR + K_e\,\omega, \qquad \tau = K_t\,I$$

(**memorize**.) K_e is the back-EMF constant (V per rad/s) and K_t the torque constant (N·m per A); in SI units they are numerically equal. The motor generates a voltage proportional to speed that opposes the supply, so current, and therefore torque, is highest at stall (ω = 0) and zero at no-load speed.

$$\tau(\omega) = \tau_{stall}\left(1 - \frac{\omega}{\omega_{nl}}\right), \qquad P_{max} = \frac{\tau_{stall}\,\omega_{nl}}{4} \text{ at } \tfrac{1}{2}\tau_{stall}$$

(**memorize**.) The torque–speed line is straight; power is a parabola peaking at half stall torque and half no-load speed.



Design rule: run the motor near 20–40% of stall torque. Peak power is at 50% but efficiency there is poor and heat builds quickly; stall current (V/R) is often 5–10× the rated continuous current and will cook the windings in seconds.

Motor types compared

Type	Controls	Torque / weight	Cost	Driver	Best for
Brushed DC (gearmotor)	Speed via voltage; position with an encoder	Medium	Low	H-bridge	Wheels, simple drives
Brushless DC (BLDC)	Speed via ESC; torque/position with FOC controller	High	Medium–high	ESC or FOC driver	Drones, high-power drives, legged joints (with FOC)
Stepper	Position open-loop, in fixed steps	High at low speed, falls fast	Low–medium	Step/dir driver (A4988, TMC2209)	3D printers, CNC, slow precise axes
Hobby servo	Position (0–180° typ.) via pulse width	Medium	Low	None (built in)	Small arms, grippers, steering
Smart serial servo (Dynamixel, ST3215)	Position, speed, torque, feedback	Medium	Medium–high	None (UART/TTL bus)	Arms, small legged robots
Linear actuator	Extension	High force	Medium	H-bridge	Lifts, doors

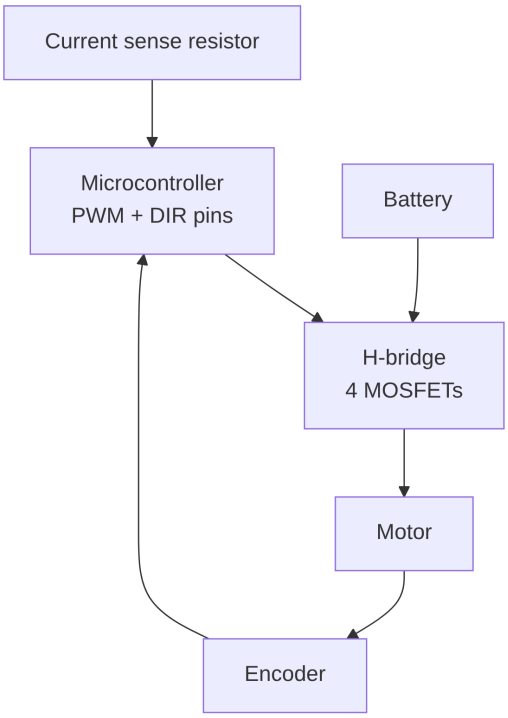
Gear ratio

$$\tau_{out} = N\,\tau_{in}\,\eta, \qquad \omega_{out} = \frac{\omega_{in}}{N}$$

(**memorize**.) A gearbox trades speed for torque. A 12 V motor at 10,000 RPM and 0.02 N·m through a 50:1 gearbox at 80% efficiency gives 200 RPM and 0.8 N·m. Reflected inertia scales with N^2, which is why high-ratio arms feel stiff and are hard to back-drive; low-ratio (≤10:1) “quasi-direct-drive” joints are what modern legged robots use so the legs can feel impacts.

H-bridges and ESCs

An H-bridge is four switches that let current flow through the motor in either direction; PWM on one pair sets speed. Datasheet numbers that matter: continuous current, peak current, logic voltage, and whether it has current sensing.



The encoder closes the position/speed loop; current sense closes a torque loop and detects stalls.

Driver	Continuous current	Notes
L298N	~1–2 A	Old bipolar design, drops 2–3 V; avoid for anything serious
TB6612FNG	1.2 A (3.2 A peak)	Efficient, tiny, ideal for small robots
DRV8871 / DRV8876	3.5 A	Single channel, current limit, good all-rounder
BTS7960 (IBT-2)	~20 A+	Cheap high-current half-bridges
Cytron MD13S / MD30C	13 A / 30 A	Reliable hobby high-current boards
VESC / ODrive / MKS-style FOC	20–60 A	Brushless with encoder feedback, torque control

Brushless motors need an **ESC** that commutates the three phases. Drone ESCs (BLHeli, AM32) are for speed only; **FOC** (field-oriented control) drivers add an encoder and give smooth torque and position control, which legged robots and arms need.

Steppers

Steps per revolution (1.8° motors = 200 full steps) times microstepping gives resolution; a 16× microstep driver gives 3,200 steps/rev. Torque falls sharply with speed because winding inductance limits how fast current can rise, so steppers are rated by holding torque and are strongest standing still. They skip steps silently when overloaded; add an encoder or end-stop homing if position matters.

Servos

Hobby servos contain a motor, gearbox, potentiometer and a control loop. Their torque rating is stall torque at rated voltage (kg·cm; divide by 10.2 for N·m). Metal-gear, 6–8.4 V "high-voltage" units are 2–4× stronger than plastic 4.8 V ones. Smart serial servos add position, load, temperature and voltage feedback over a daisy-chained bus, which is why they dominate hobby arms and small quadrupeds.

Thermal reality

Motor torque ratings assume the copper stays cool. Continuous torque is set by I^2R heating; peak torque (2–3× continuous) is available for seconds. If a joint has to hold a load against gravity for long periods, either gear it so the holding current is small, add a brake or counterbalance, or accept that the motor will get hot and derate.

Mechanical engineering: gears, stress, structures, materials

Mechanics decides whether the robot stays rigid, moves smoothly and survives being dropped. The two skills are transmitting torque without losing it and keeping deflection small without adding weight.

Transmissions

Type	Ratio range per stage	Efficiency	Backlash	Notes
Spur gears	1:1 – 6:1	95–98%	Low–medium	Simple, printable, noisy at speed
Planetary gearbox	3:1 – 10:1	90–97%	Low	Compact, coaxial, stacks to 100:1+
Worm gear	10:1 – 100:1	40–70%	Low	Self-locking (holds loads unpowered), not back-drivable
Timing belt (GT2, HTD)	1:1 – 5:1	95–98%	Near zero	Quiet, spans distance, needs tensioning
Cycloidal / harmonic	10:1 – 160:1	70–90%	Very low	High torque density; used in arm joints
Lead screw / ball screw	Linear	30–50% / 85–95%	Low	Converts rotation to linear force

Gear ratio in teeth (**memorize**): N = T_output / T_input. Belt ratio = pulley tooth counts the same way. Lead-screw force (**memorize**):

$$F = \frac{2\pi\,\tau\,\eta}{L}$$

with L the lead (mm advanced per turn). A 0.4 N·m stepper on an 8 mm lead screw at 40% efficiency pushes about 126 N.

Stress, strain and deflection

$$\sigma = \frac{F}{A}, \qquad \varepsilon = \frac{\Delta L}{L}, \qquad \sigma = E\,\varepsilon$$

(**memorize**.) Stress is force per area; strain is fractional stretch; E (Young's modulus) is stiffness. A part fails when σ exceeds the material's yield strength; it deflects too much when E is low or the section is thin. Most robot parts fail in **bending**, not tension, so the cantilever formula is the one to keep (**memorize**):

$$\delta_{tip} = \frac{FL^3}{3EI}, \qquad \sigma_{max} = \frac{Mc}{I}, \qquad I_{rect} = \frac{bh^3}{12}$$

Read the exponents: doubling length gives 8× deflection; doubling thickness (h) gives 1/8. That is why tubes and I-beams beat solid rods per gram, and why a 20 cm arm is not just twice as floppy as a 10 cm one.

Safety factor and fatigue

Design so that allowable stress = yield strength / safety factor, with a factor of 2–3 for static loads and 4+ for impacts or repeated cycling. Metal survives many cycles below its fatigue limit; 3D-printed plastic does not have one, so printed parts under repeated load will eventually crack.

Materials

Material	Density (g/cm³)	E (GPa)	Yield (MPa)	Machinability	Notes
PLA (printed)	1.24	3.5	50	Easy	Stiff, brittle, softens at 55 °C
PETG (printed)	1.27	2.1	45	Easy	Tougher, some flex, good default
ABS / ASA (printed)	1.05	2.0	40	Easy	Heat-resistant, warps; ASA is UV-stable
Nylon / PA-CF (printed)	1.1	3–8	50–100	Medium	Tough, wear-resistant, absorbs water
Acrylic (laser cut)	1.18	3.2	70	Easy	Clear, brittle, cracks at screws
Plywood / MDF	0.6–0.75	4–10	30–50	Easy	Cheap chassis material
Delrin (POM)	1.41	3.0	65	Easy	Low friction, great for bushings and gears
Aluminum 6061-T6	2.70	69	276	Easy	The default metal; extrusions, plates

Material	Density (g/cm³)	E (GPa)	Yield (MPa)	Machinability	Notes
Steel (mild)	7.85	200	250	Medium	Shafts, brackets, cheap and strong
Steel (hardened, drill rod)	7.85	200	600+	Hard	Precision shafts, pins
Carbon fiber tube	1.6	70–140 (along fiber)	500+	Dusty, hazardous	Highest stiffness/weight; drone arms, legs

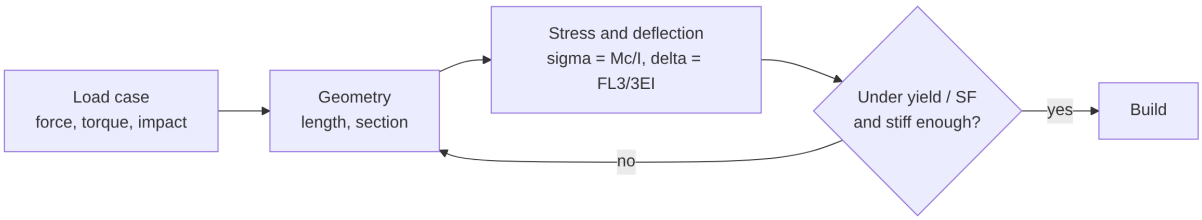
Specific stiffness (E/density) is the number to compare for weight-limited parts: carbon ≈ 60, aluminum ≈ 26, steel ≈ 25, PLA ≈ 2.8. Aluminum and steel are equally stiff per gram; carbon is more than twice as good; plastics are an order of magnitude worse, so printed parts need thick sections and ribs.

Fasteners and joints

- Metric hardware (M2, M2.5, M3, M4, M5) covers nearly all hobby robotics; M3 is the workhorse. A screw's clamping force, not its shear strength, is what holds parts.
- Use nylock nuts or threadlocker anywhere there is vibration; heat-set inserts for printed parts; T-nuts for aluminum extrusion.
- Shafts: D-shafts with set screws for low torque, keyed or flatted shafts with clamp couplings for high torque, shaft collars to control axial play.
- Bearings: radial ball bearings (608, 625, 6800 series) for rotating loads; flanged for easy mounting; thrust bearings for axial loads; plastic bushings where speed is low.

Structural intuition

Triangles are rigid, rectangles are not; brace every frame diagonally. Put mass low and centered. Mount motors so the shaft is supported near the load (a bearing beside the wheel, not just the gearbox's own bearing). Design so that the part that breaks is the one that is cheap and easy to replace.



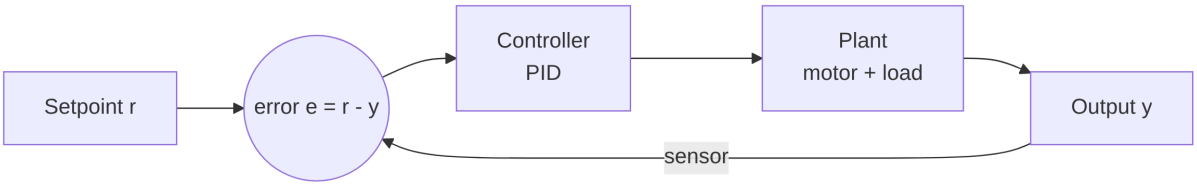
Iterating on geometry, not material, is usually the cheapest fix: a thicker section beats a stronger alloy.

Control: feedback, PID, tuning, state estimation

Control is the art of making the robot do what you meant despite friction, load and noise. Feedback (measure, compare, correct) is what lets a cheap motor act like a precise one. PID handles 90% of hobby and a large share of professional robots.

Open loop vs closed loop

Open loop: command a PWM and hope. Closed loop: measure the result, compute the error, adjust. The first fails the moment the battery sags or the floor changes; the second self-corrects.



The minus sign at the summing point is the whole idea: the correction opposes the error.

PID

$$u(t) = K_p\, e(t) + K_i \int_0^t e\, d\tau + K_d \frac{de}{dt}$$

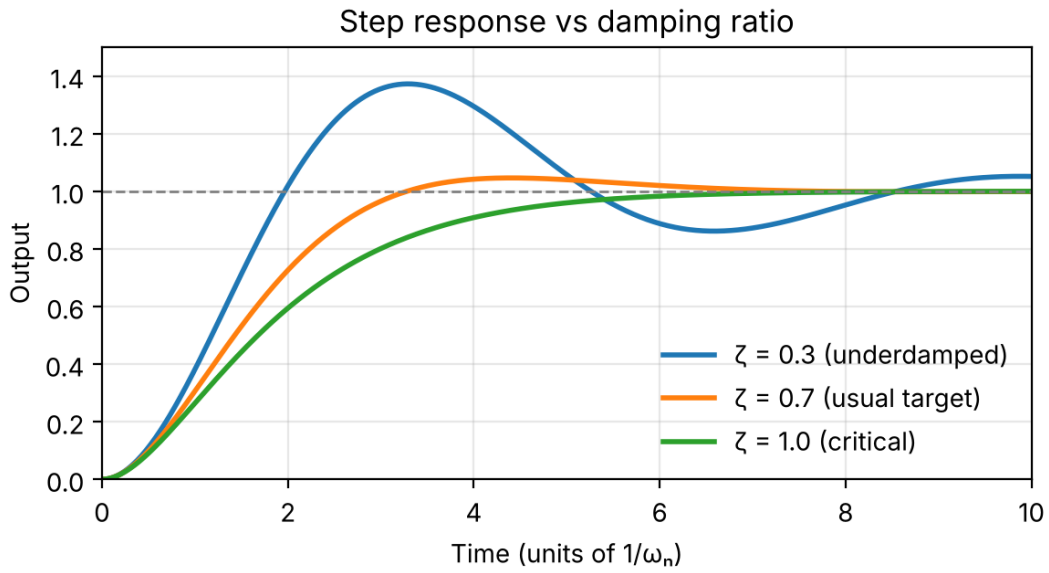
(**memorize**.) Discrete form your code runs (**memorize**):

$$u_k = K_p e_k + K_i\, \Delta t \sum_{j \le k} e_j + K_d \frac{e_k - e_{k-1}}{\Delta t}$$

Term	Reacts to	Increases	Side effect
P	Present error	Speed of response	Oscillation, steady-state error remains
I	Accumulated error	Removes steady-state error (gravity, friction)	Overshoot, windup if actuator saturates
D	Rate of change	Damping, less overshoot	Amplifies noise; needs a filtered derivative

Metaphor: P is a spring pulling you to the target, D is a shock absorber, I is a slow hand that notices you keep stopping short and nudges the target.

What tuning changes



Raising K_p raises the natural frequency and lowers damping (more like the ζ = 0.3 curve); raising K_d restores damping. Aim for ζ ≈ 0.7: about 5% overshoot and the fastest settling.

Practical tuning recipe

1. Set K_i = K_d = 0. Raise K_p until the system oscillates steadily; call that K_u and the period T_u.
2. Ziegler–Nichols starting point: K_p = 0.6 K_u, K_i = 1.2 K_u / T_u, K_d = 0.075 K_u T_u. Then halve K_i and tune by feel.
3. Add D first if you see overshoot; add I only if there is a persistent offset.
4. Clamp the integral (anti-windup) at the actuator limit; a joint pushed against a wall will otherwise wind up and lunge when released.
5. Filter the derivative input (EMA from Section 6) or differentiate the measurement instead of the error to avoid a kick on setpoint changes.

Loop rate: at least 10× the system's bandwidth. A wheel speed loop at 100 Hz–1 kHz, a current loop at 10–20 kHz, a drone attitude loop at 500 Hz–8 kHz, a balancing robot at 200–1000 Hz.

Feedforward

Feedback fixes errors after they happen; feedforward prevents them. If you know the torque gravity puts on a joint (m g L cos θ), add it to the command directly and let PID handle only the residual. For velocity control add K_v × desired speed. Feedforward + a modest PID beats aggressive PID every time.

Cascaded loops

Motor control stacks loops from fast to slow: an inner current (torque) loop, a speed loop around it, a position loop around that. Each outer loop's output is the inner loop's setpoint. This is how FOC drivers, industrial servos and modern legged robots are structured.

State estimation

You rarely measure what you want to control. Estimation combines a model (how the state evolves) with measurements (noisy, delayed).

- **Dead reckoning / odometry**: integrate wheel encoders (Section 11). Precise short term, drifts without bound.
- **Complementary filter**: Section 6; good enough for tilt and heading.
- **Kalman filter**: the optimal blend when you can state the model and noise as Gaussians. Predict with the model, then correct toward the measurement by a gain K that is large when the sensor is trustworthy and small when the model is (**memorize** the shape):

$$\hat{x}_k = \hat{x}_k^- + K_k\,(z_k - H\hat{x}_k^-)$$

The complementary filter is a Kalman filter with a fixed gain. Start there; move to an EKF when you have multiple sensors of different quality (GPS + IMU + wheels).

Beyond PID (know the names)

LQR (optimal gains for a linear model; the usual choice for balancing robots and drones), MPC (optimizes a trajectory over a horizon every step; used in legged locomotion), and learned policies (reinforcement learning, increasingly common for quadrupeds). All of them still sit on top of the same current and position loops.

Kinematics: forward and inverse kinematics for arms and legs

Forward kinematics (FK) asks "given the joint angles, where is the hand?"; inverse kinematics (IK) asks "to put the hand there, what angles?" FK is always a straightforward matrix chain. IK is the interesting part, and a leg is just an arm whose "hand" is a foot.

Forward kinematics

For a planar two-link arm with lengths L_1, L_2 and joint angles θ_1 (shoulder, from horizontal) and θ_2 (elbow, relative to link 1) (**memorize**):

$$x = L_1\cos\theta_1 + L_2\cos(\theta_1+\theta_2), \qquad y = L_1\sin\theta_1 + L_2\sin(\theta_1+\theta_2)$$

Each term is one link resolved into x and y; the elbow angle adds to the shoulder angle because link 2 is measured relative to link 1. For any number of links in 3D the same idea is written as a product of homogeneous transforms (Section 2): $T = T_1(\theta_1)\cdot T_2(\theta_2)\cdot\ldots\cdot T_n(\theta_n)$.

Denavit–Hartenberg parameters

DH is a bookkeeping convention that describes each joint-to-joint transform with four numbers, so any serial arm becomes a table.

Symbol	Meaning
θ	Rotation about the previous z axis (the joint variable for a revolute joint)
d	Offset along the previous z axis (the joint variable for a prismatic joint)

Symbol	Meaning
a	Link length along the new x axis
α	Twist: rotation about the new x axis

$$T_i = R_z(\theta_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,R_x(\alpha_i)$$

You do not need to memorize the expanded 4×4; you need to know the order and be able to fill the table for your own arm. Many people skip DH and just write the transforms directly from the CAD model, which is fine.

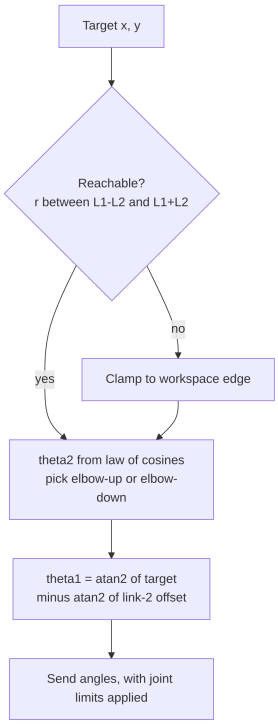
Inverse kinematics: analytic (two-link)

For the planar two-link arm, reach the point (x, y) with the law of cosines (**memorize** the whole recipe):

$$\cos\theta_2 = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

$$\theta_2 = \pm\arccos(\cdot), \qquad \theta_1 = \mathrm{atan2}(y, x) - \mathrm{atan2}(L_2\sin\theta_2,\ L_1 + L_2\cos\theta_2)$$

The ± is the elbow-up / elbow-down choice: two solutions for every reachable point. If $|\cos\theta_2| > 1$ the target is out of reach (further than $L_1 + L_2$ or closer than $|L_1 - L_2|$). This two-link solution is the core of nearly every hobby arm and every leg.



Always clamp before solving; an unreachable target produces NaN, and NaN sent to a servo is a lunge.

Adding a base rotation (3-DOF arm or leg)

A yaw joint at the base turns a 3D target (x, y, z) into a planar problem: $\theta_0 = \text{atan2}(y, x)$, then solve the two-link problem with horizontal reach $r = \sqrt{x^2 + y^2}$ and height z. Three lines of code give you a 3-DOF arm or a quadruped leg (hip abduction, hip pitch, knee).

Legs: the same math, upside down

A quadruped leg is a two-link arm whose base is the hip and whose tip is the foot. The body is the fixed frame, the foot target is a point below and slightly ahead or behind; the solver above returns hip-pitch and knee. With a hip abduction joint, first rotate the target into the leg's plane (the base-rotation trick) and add a hip offset. For a 3-DOF hexapod leg (coxa, femur, tibia) it is identical: coxa angle from atan2, then two-link IK.

The extra requirement for legs is that the **foot trajectory** is planned in the body frame (a swing arc up-and-forward, a stance line backward) and IK runs every control tick on that moving target (Section 11).

Numerical IK: the Jacobian

For arms with more joints than the analytic solution handles, use the Jacobian J, the matrix of partial derivatives $\partial x/\partial \theta$ (**memorize** the relation):

$$\dot{\mathbf{x}} = J(\theta)\,\dot{\theta}, \qquad \Delta\theta = J^{+}\,\Delta\mathbf{x}$$

Iterate: compute the position error Δx, multiply by the pseudo-inverse J^+ (or J^T scaled by a small gain, which is gradient descent and surprisingly robust), update θ, repeat until the error is small. Near a singularity ($\det J \to 0$, arm fully extended) the required joint speeds blow up; damped least squares ($J^+ = J^T(JJ^T + \lambda^2 I)^{-1}$) tames it. Metaphor: the Jacobian is the arm's local "exchange rate" between joint motion and hand motion.

The Jacobian also maps forces the other way (**memorize**): joint torques $\tau = J^T F$. This is how a legged robot converts a desired ground-reaction force at the foot into hip and knee torques, and how an arm's motor currents estimate the load in its hand.

Workspace and joint limits

Plot the reachable set once for any arm you build (sweep the angles, run FK, scatter the points). Keep targets away from the boundary, where the arm is nearly straight and both sensitivity and required torque peak. Always apply joint limits after IK, never before, so the solution stays consistent.

Trajectory basics

Never jump a joint to its target. Interpolate: linear for quick tests, cubic or quintic polynomials for smooth velocity and acceleration, or a trapezoidal velocity profile when the joint has a speed limit. A minimum-jerk quintic from θ_0 to θ_1 over time T:

$$\theta(t) = \theta_0 + (\theta_1 - \theta_0)\left(10s^3 - 15s^4 + 6s^5\right), \quad s = t/T$$

This single polynomial makes servo arms look professional.

Mobile robots, legged locomotion and drones

Locomotion is where kinematics, control and physics meet the floor. Wheels are the easiest; legs reuse arm IK plus timing; drones are an attitude-control problem with thrust.

Differential drive

Two driven wheels, wheelbase b (distance between them), wheel radius r. With wheel angular speeds ω_L and ω_R (**memorize**):

$$v = \frac{r(\omega_R + \omega_L)}{2}, \qquad \omega = \frac{r(\omega_R - \omega_L)}{b}$$

Inverse (what to command for a desired v and ω) (**memorize**):

$$\omega_R = \frac{v + \omega b/2}{r}, \qquad \omega_L = \frac{v - \omega b/2}{r}$$

Odometry integrates encoder distance per tick, Δs_L and Δs_R (**memorize**):

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \quad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{b}, \quad x \mathrel{+}= \Delta s\cos\theta,\ y \mathrel{+}= \Delta s\sin\theta,\ \theta \mathrel{+}= \Delta\theta$$

Use θ at the midpoint of the step for better accuracy. Odometry drifts from wheel slip and a wrong b; calibrate b by driving a full rotation and comparing the heading. Fuse with a gyro (complementary filter) for heading; wheels for distance.

Other wheel layouts

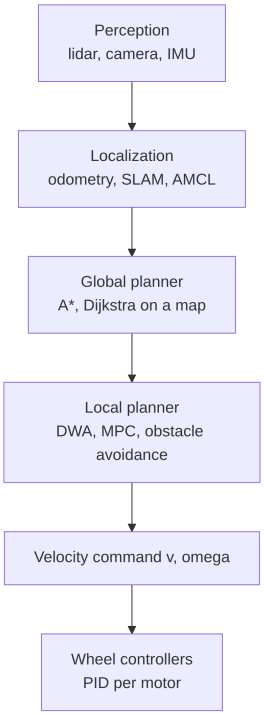
Layout	Motion	Notes
Differential (2 wheels + caster)	Forward, turn in place	Simplest; the default
Skid steer (4 or 6 wheels, tank)	Same, by slipping	Robust, odometry poor on turns
Ackermann (car steering)	Cannot turn in place	Fast, needs steering servo and path planning
Mecanum (4 rollers at 45°)	Holonomic: slides sideways	Needs flat floor; sums four wheel speeds
Omni (3 wheels at 120°)	Holonomic	Compact, lower traction

Mecanum wheel speeds for body velocities (vx forward, vy left, ω yaw), with L and W the half-length and half-width (**memorize** the pattern):

$$\omega_{FL} = \tfrac{1}{r}(v_x - v_y - (L+W)\omega), \quad \omega_{FR} = \tfrac{1}{r}(v_x + v_y + (L+W)\omega)$$

$$\omega_{RL} = \tfrac{1}{r}(v_x + v_y - (L+W)\omega), \quad \omega_{RR} = \tfrac{1}{r}(v_x - v_y + (L+W)\omega)$$

Navigation stack (know the layers)



ROS 2 Nav2 implements this whole stack; understanding each box is what lets you configure it rather than copy a launch file.

Legged locomotion

A leg cycle has a **stance** phase (foot on ground, pushing the body forward) and a **swing** phase (foot in the air, returning). The **duty factor** is the fraction of the cycle in stance. Gaits are patterns of which legs are in stance:

Gait (quadruped)	Legs in stance	Duty factor	Speed	Stability
Crawl / walk	3 at a time	≥ 0.75	Slow	Statically stable
Trot	Diagonal pairs	~0.5	Medium	Dynamically stable
Pace	Lateral pairs	~0.5	Medium	Rolls
Bound / gallop	2 then 2, flight phase	< 0.5	Fast	Dynamic, needs control

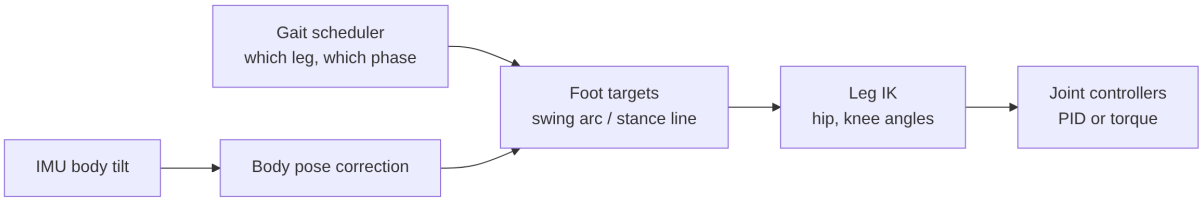
Static stability: the vertical projection of the center of mass stays inside the support polygon (the convex hull of the feet on the ground). A crawl keeps three feet down, so the polygon is a triangle and the robot can pause at any instant. **Dynamic stability** (trot, running) lets the CoM leave the polygon briefly, relying on the next foot placement to catch it.

The simplest useful dynamic model is the **linear inverted pendulum**: body of mass m at height h on a massless leg. Its natural time constant (**memorize**):

$$T_c = \sqrt{h/g}$$

A 0.3 m tall body has $T_c \approx 0.175$ s, so it must take steps faster than a few times that to stay up. Raibert's foot placement heuristic (**memorize**): place the foot at $x_{foot} = v \cdot T_{stance}/2 + k \cdot (v - v_{desired})$. The first term is neutral, the second accelerates or brakes. This one formula, with a stiff leg and IK per tick, walks a hobby quadruped.

Swing trajectory: a half-ellipse or Bézier curve from lift-off to touchdown, height 2–5 cm; run IK on the curve at 100–500 Hz. Ground reaction forces per foot are balanced so their sum equals mg and their moments cancel; with a torque-controlled leg use $\tau = J^T F$ from Section 10.



Every hobby quadruped is this five-box loop; a research one adds an MPC or a learned policy in place of the scheduler.

Bipeds and balancing robots

A two-wheeled balancing robot is the inverted pendulum with wheels: measure tilt (complementary filter), PID or LQR to a wheel torque that keeps the tilt at zero, and a slow outer loop that leans the robot to move. Bipeds add the zero-moment point (ZMP), the point on the ground where net moment is zero; keep it inside the foot and the robot does not tip.

Drones, briefly

A quadcopter has four rotors; total thrust sets vertical acceleration, differential thrust sets roll, pitch and yaw. Hover requires thrust = weight (**memorize**): $T_{total} = mg$, so each motor gives $mg/4$, and you want hover at roughly 50% throttle for control margin (thrust-to-weight $\approx 2{:}1$).

- Attitude loop: rate PID on gyro, angle loop around it, 500 Hz–8 kHz, all on the flight controller (Betaflight, PX4, ArduPilot).
- Position: GPS + barometer outdoors, optical flow or motion capture indoors.
- Motor sizing: kV × battery volts = no-load RPM; propeller thrust rises with RPM^2 and $diameter^4$, so bigger, slower props are more efficient.
- Build on an existing flight controller; roll your own only for the learning, not the flying.

Embedded software and system integration

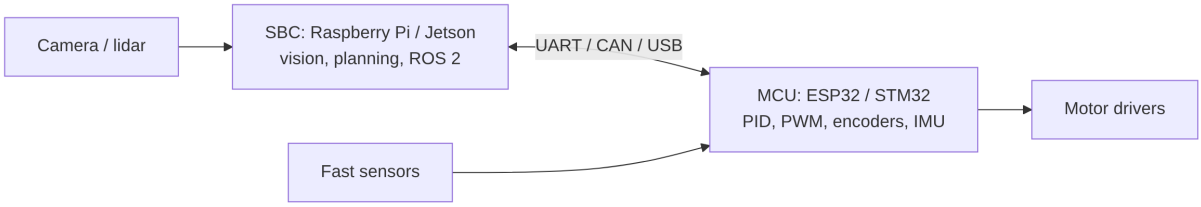
Robot code is a set of loops running at fixed rates, not a script that runs top to bottom. Getting timing right matters more than any algorithm.

Compute tiers

Tier	Examples	Runs	Strength
Microcontroller (MCU)	ESP32, RP2040/Pico, STM32, Teensy 4.x, Arduino	Bare metal or RTOS	Hard real-time: PWM, encoders, PID at kHz rates
Single-board computer (SBC)	Raspberry Pi 5, Jetson Orin Nano, Radxa	Linux, ROS 2	Perception, planning, networking, cameras

Tier	Examples	Runs	Strength
Motor controllers with MCUs	ODrive, VESC, Dynamixel, SimpleFOC boards	Firmware	Offload current/velocity loops

The standard split: an MCU owns every time-critical loop and talks to an SBC over serial or CAN; the SBC does everything slow and smart. Do not run a 1 kHz control loop on Linux and expect it to stay at 1 kHz.



The boundary is the loop rate: anything that must happen every millisecond lives on the left.

Timing and the control loop

A fixed-rate loop (**memorize** the pattern):

```
next = now()
loop:
  read sensors
  estimate state
  compute control
  write actuators
  next += DT
  sleep_until(next)
```

Use a hardware timer or `sleep_until`, never `delay(DT)` after variable work, or the period drifts with load. Measure your loop: print max execution time occasionally; if it approaches DT you are about to have jitter. Never block inside the loop (no waiting on serial, no long I²C transactions, no floating-point-heavy printing).

Interrupts handle encoders and time-critical inputs; keep the ISR to a few lines (increment a count, set a flag) and mark shared variables `volatile`. Read multi-byte shared values with interrupts briefly disabled or via atomic copies.

Units and numeric hygiene

- Decide the unit of every variable and put it in the name: `speed_mps`, `angle_rad`, `torque_nm`. Most integration bugs are unit bugs.
- Use `float` on MCUs with an FPU (ESP32, STM32F4+, Teensy); avoid `double` there. Use integers for encoder counts (a 32-bit signed counter at 4000 counts/rev overflows after 536,000 revs; fine, but know it).
- Wrap angles to $(-\pi, \pi]$ after every subtraction; the difference between 179° and −179° is 2°, not 358°.

Communication and protocols

Between boards, send fixed-size binary packets with a start byte, length and checksum, not free-form text. Between the robot and your laptop, use a serial console for debugging and a proper transport (ROS 2 topics over Wi-Fi, MQTT, or a WebSocket) for telemetry.

Safety first, always

- Hardware emergency stop that cuts motor power, independent of software.
- Watchdog: if the SBC stops sending commands for 200 ms the MCU stops the motors.
- Startup in a safe state: outputs off until initialization completes and the first valid command arrives.
- Current and temperature limits in firmware; joint limits enforced before anything reaches a driver.

Simulation and tooling

Simulate first when the robot is expensive or can hurt you. Gazebo and MuJoCo for physics, PyBullet for quick scripts, Isaac Sim / Isaac Lab for learned policies, Webots as a friendly all-in-one. Model the robot once in URDF and reuse it for simulation, ROS 2 and visualization. For MCU code, unit-test the math (IK, filters, PID) on the desktop; only the hardware layer needs the hardware.

Debugging like an engineer

1. Reproduce reliably.
2. Isolate a layer: does the encoder count correctly by hand-turning the wheel? Does open-loop PWM move the motor? Then close the loop.
3. Log time-stamped data and plot it; a graph of setpoint vs measurement diagnoses a PID in seconds where staring at the robot never will.
4. Change one variable at a time and write down what happened.

This four-step ritual is the practical form of “escaping tutorial hell”: you stop hoping and start measuring.

Formulas to memorize, project ladder, further reading

These 30 formulas are the ones drilled in the exercise book. If you can write each from memory and say in one sentence what question it answers, you can size, wire, control and move almost any hobby-to-prosumer robot.

#	Formula	Answers
1	$a \cdot b = \|a\|\|b\| \cos\theta$	How aligned are two directions?
2	$a \times b$, $\|a \times b\| = \|a\|\|b\| \sin\theta$	Torque axis, perpendicular vector
3	$c^2 = a^2 + b^2 - 2ab\cos C$	Two-link IK elbow angle
4	atan2(y, x)	Heading of a vector, all quadrants
5	$R(\theta) = [[\cos, -\sin],[\sin, \cos]]$	Rotate a vector or frame
6	$T = [R\ p;\ 0\ 1]$, $T_{chain} = T_1T_2\ldots T_n$	Frame to frame, base to tool
7	d/dt rules; $\dot{x} \approx \Delta x/\Delta t$; X += x·Δt	Rates and accumulation in code
8	y(t) = u(1 − e^{−t/τ})	First-order settling, time constant

#	Formula	Answers
9	$\omega_n = \sqrt{(k/m)}$, $\zeta = c / 2\sqrt{(km)}$	Oscillation and damping
10	$v = v_0 + at$; $x = x_0 + v_0 t + \frac{1}{2}at^2$	Motion under constant acceleration
11	$v = \omega r$; $\omega = 2\pi \cdot RPM/60$	Wheel speed from motor speed
12	$\Sigma F = ma$; $\Sigma \tau = I\alpha$	Force and torque needed
13	$\tau = rF \sin \theta$	Torque from a lever
14	$I = I_c + md^2$	Inertia about a shifted axis
15	$P = Fv = \tau\omega$; $E = Pt$	Motor power, battery energy
16	$\eta_{total} = \eta_1\eta_2\eta_3$	Losses along a drivetrain
17	$F_f = \mu N$	Traction limit
18	$V = IR$; $P = VI = I^2R$	Everything electrical
19	R series / parallel; $V_{out} = V_{in} R_2/(R_1+R_2)$	Dividers and networks
20	$t = Ah / A$; $Wh = V \times Ah$	Battery run time
21	$code = V/V_{ref} \cdot (2^N - 1)$	ADC reading
22	$V_{avg} = duty \times V_{supply}$	PWM
23	$y_k = \alpha x_k + (1-\alpha) y_{k-1}$	Low-pass filter
24	$\theta = \alpha(\theta + \omega\Delta t) + (1-\alpha)\theta_{accel}$	Complementary filter
25	$V = IR + K_e \omega$; $\tau = K_t I$; $\tau(\omega)$ linear	DC motor behaviour
26	$\tau_{out} = N\tau_{in}\eta$; $\omega_{out} = \omega_{in}/N$	Gearing
27	$\sigma = F/A = E\varepsilon$; $\delta = FL^3/3EI$; $I = bh^3/12$	Will it bend or break?
28	$u = K_p e + K_i \int e + K_d \dot{e}$	PID
29	Two-link FK and IK; $\dot{x} = J\theta$; $\tau = J^T F$	Arms and legs
30	$v = r(\omega_R+\omega_L)/2$; $\omega = r(\omega_R-\omega_L)/b$; odometry update	Wheeled robots

Project ladder

Each step should be designed, wired, coded and tuned by you, with a one-page write-up of the numbers you computed and the ones you measured.

1. **Encoder + PID speed loop** on one gearmotor. Plot setpoint vs measured speed. Skill: Sections 6, 7, 9.
2. **Differential-drive robot** that drives a 1 m square using odometry only, then with gyro heading. Skill: Section 11.
3. **Two-link arm** with servos and analytic IK drawing a circle. Skill: Section 10.
4. **Balancing robot** with a complementary filter and PID. Skill: Sections 6, 9, 11.
5. **Quadruped** (8 or 12 servos) that crawls, then trots, with IK per leg and a gait scheduler. Skill: everything so far.
6. **Brushless FOC joint** (e.g. SimpleFOC or ODrive) with a torque-controlled leg that can hop. Skill: Sections 7, 9, 10.
7. **Mobile manipulator** running ROS 2 Nav2 with a camera and an arm. Skill: Section 12 plus perception.

Habits that build intuition

- Estimate every quantity before measuring it; keep a running log of how far off you were.
- Read one datasheet fully per week (a motor, a driver, a sensor).
- Rebuild one thing you already made, but 2× the size or 2× the speed, and see what breaks.

- Explain each formula's "why" to someone else; the gaps you find are the next thing to learn.

Further reading

- *Modern Robotics* (Lynch & Park): kinematics and dynamics, free online with videos.
- *Introduction to Robotics: Mechanics and Control* (Craig): the classic arm textbook.
- *Practical Electronics for Inventors* (Scherz & Monk): the electronics half of this guide, expanded.
- *Shigley's Mechanical Engineering Design*: stress, fasteners, gears.
- *Feedback Systems* (Åström & Murray): control theory, free online.
- Brian Douglas's control videos and Steve Brunton's data-driven control series (YouTube).
- SimpleFOC, ODrive and Open Dynamic Robot Initiative documentation for modern actuator design.